



UNIVERSITÀ DEGLI STUDI DI ROMA "LA SAPIENZA"

DIPARTIMENTO DI INGEGNERIA INFORMATICA, AUTOMATICA E GESTIONALE
"ANTONIO RUBERTI"

Master in AI and Robotics - Autonomous and Mobile Robotics

Single Rigid Body MPC for Humanoid Locomotion

Authors:

Andrea D'ALESSIO (2208280)
Francesco BARBETTA (1991907)
Marco Antonio IOSSA (2001300)

Professor/Supervisors:

Prof. Giuseppe ORIOLO
Dr. Nicola SCIANCA
Dr. Eugenio BUGLI

A.Y. 2025/2026

Contents

1	Introduction	2
1.1	Project Objectives	2
1.2	Organization	3
2	Mathematical And Dynamic Modeling	4
2.1	The Single Rigid Body Dynamics (SRBD) Model	4
2.2	Orientation Representation Via Quaternions	5
2.3	Coupling SRBD Rotational Dynamics And Quaternion Kinematics	6
2.4	Rotational Dynamics And Inertia Tensor Projection	6
3	Control Architecture And Implementation	7
3.1	System Overview And Architecture	7
3.2	Footstep Planning And Swing Trajectories Generation	8
3.2.1	Discrete Footstep Planning	8
3.2.2	Continuous Swing Trajectory Generation	8
3.3	Model Predictive Control (SRBD-MPC)	9
3.3.1	State And Control Variables	9
3.3.2	Discretization And Prediction Horizon	10
3.3.3	Cost Function Formulation	10
3.3.4	Physical And Kinematic Constraints	11
4	Whole-Body Control (WBC) And Inverse Dynamics	13
4.1	QP Problem Formulation	13
4.2	Knee Anti-Hyperextension Guard	14
5	Advanced Disturbance Recovery Strategies	15
5.1	Reactive Footstep Replanning (MPC Level)	15
5.2	Dynamic Cost Weight Scheduling (Replanning Mode)	15
5.3	Kinematic Momentum Control (Dynamic Arm Swing)	15
5.4	Numerical Reliability: Safe Fallback And Warm-Starting	16
6	Experiments And Results Analysis	17
6.1	Testing Environment, Simulation Setup And Robot Physical Parameters	17
6.2	Disturbance Rejection And Real-Time Trajectory Readaptation Vs Baseline Behavior	18
6.3	Ablation Of Impact Intensity And Compliance	25
7	Conclusions And Future Work	27
8	References	28

Chapter 1

Introduction

The realization of stable and dynamic bipedal locomotion in anthropomorphic robots represents one of the most significant challenges in modern robotics. The non-linear and high-dimensional nature of the robot’s dynamics, coupled with the inherent instability of a bipedal configuration, demands sophisticated control strategies capable of operating in real-time. Traditional approaches relying on full-order dynamics often require excessive computational power, hindering their application in fast, reactive control loops.

To bridge this gap, dynamic model simplification becomes essential. The Single Rigid Body Dynamics (SRBD) model offers a computationally efficient approximation by treating the robot’s mass as concentrated at the Center of Mass (CoM) and neglecting internal dynamics, thereby simplifying the complex equations of motion while retaining the essential dynamics governing overall balance and reaction forces. This simplification is crucial for enabling the high-frequency decision-making required for dynamic walking.

1.1 Project Objectives

This project aims to design, implement, and test an advanced Model Predictive Control (MPC) framework based on the Single Rigid Body Dynamics (SRBD) approximation for humanoid locomotion. The starting point for this work was an existing reference codebase provided for the project: the IS-MPC framework [1], originally developed in Python using the dartpy physics engine [2]. This foundational code successfully handled humanoid gait generation but relied on the standard Linear Inverted Pendulum (LIP) model, which inherently ignores the robot’s rotational inertia.

To overcome these limitations and achieve highly robust balance recovery, this project draws direct inspiration from the SRBD-based control architecture proposed by Rossini et al. [3]. While their complete framework includes dynamic obstacle avoidance, our work focuses squarely on extracting and implementing the core non-linear SRBD MPC formulation and reactive footstep replanning to ensure real-time physical stability against unexpected impacts.

Specifically, the project focused on:

- **Upgrading the dynamic model:** Transitioning from the point-mass assumption of the LIP to the SRBD model to actively exploit the robot’s rotational inertia for balance.
- **Integrating a basic Footstep Planner:** Coupling the MPC with a discrete footstep planner to generate a reference walking trajectory.
- **Implementing Quaternion-based Attitude Control:** Using singularity-free quaternion algebra to strictly control the robot’s base orientation (roll, pitch, yaw).
- **Validating Reactive Disturbance Rejection:** Testing the framework’s capability to dynamically replan footsteps (dynamic recovery) and modulate Ground Reaction Forces (GRFs) to prevent the robot from falling when subjected to severe, unexpected external impacts, without explicitly modeling obstacle avoidance logic.

1.2 Organization

The remainder of this report details the theoretical formulation and practical implementation of the proposed framework.

- **Chapter 2** establishes the mathematical foundations, outlining the SRBD model and introducing quaternion algebra for a singularity-free representation of rotational dynamics.
- **Chapter 3** describes the complete control architecture, which encompasses the reference footstep planner, the non-linear MPC formulated via CasADi/IPOPT, and the Inverse Dynamics Whole-Body Control (WBC).
- **Chapter 4** focuses on Whole-Body Control (WBC) and Inverse Dynamics formulations.
- **Chapter 5** covers advanced disturbance recovery strategies.
- **Chapter 6** evaluates the framework’s real-time performance through simulations in the dartpy environment with the HRP4 humanoid, focusing on nominal tracking and the system’s robustness against severe lateral impacts.
- **Chapter 7** summarizes the project’s achievements and discusses potential future directions, such as the integration of dynamic obstacle avoidance.

Chapter 2

Mathematical And Dynamic Modeling

The dynamics of a fully articulated humanoid robot are characterized by high-dimensional, highly non-linear Newton-Euler equations of motion. Incorporating the full whole-body dynamics into an online Receding Horizon Control loop is computationally prohibitive for high-frequency real-time execution. To achieve a computationally tractable Optimal Control Problem (OCP) while overcoming the physical limitations of the standard Linear Inverted Pendulum (LIP) model, which treats the robot as a point mass and ignores rotational inertia, this framework adopts the Single Rigid Body Dynamics (SRBD) approximation.

2.1 The Single Rigid Body Dynamics (SRBD) Model

The SRBD model approximates the entire multi-link humanoid structure as a single rigid body. It operates on the assumption that the total mass m of the robot is concentrated at its Center of Mass (CoM) $\mathbf{r} \in \mathbb{R}^3$, and that its centroidal inertia tensor $I_{body} \in \mathbb{R}^{3 \times 3}$ remains constant in the local body-fixed frame, computed a priori when the robot is in a nominal standing configuration. Furthermore, the orientation of this single rigid body is assumed to coincide with the base frame orientation of the robot (attached to the torso), significantly simplifying the mapping between the simplified model and the full-body kinematics.

Under these assumptions, the whole-body motion is reduced to the Centroidal Dynamics equations, which link the variation of the linear momentum $\mathbf{P} \in \mathbb{R}^3$ and angular momentum $\mathbf{L} \in \mathbb{R}^3$ to the sum of the contact forces generated at the feet among all the 8 contact points considered, 4 points per foot (square-shaped), one per corner.

The SRBD equations of motion are formally written as:

- **Linear Momentum Dynamics (Translational):**

$$\dot{\mathbf{P}} = m\ddot{\mathbf{r}} = \sum_{i=1}^8 \mathbf{f}_i + m\mathbf{g} \quad (2.1)$$

- **Angular Momentum Dynamics (Rotational):**

$$\dot{\mathbf{L}} = I_{world}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (I_{world}\boldsymbol{\omega}) = \sum_{i=1}^8 (\mathbf{p}_i - \mathbf{r}) \times \mathbf{f}_i \quad (2.2)$$

Where:

- $\mathbf{g} \in \mathbb{R}^3$ is the gravity vector.
- $\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity vector of the base frame.
- $\mathbf{p}_i \in \mathbb{R}^3$ represents the Cartesian position of the i -th foot contact point.
- $\mathbf{f}_i \in \mathbb{R}^3$ is the pure 3D ground reaction force (GRF) exerted by the i -th foot.

By manipulating these equations, the non-linear MPC explicitly solves for the linear acceleration $\ddot{\mathbf{r}}$ and the angular acceleration $\dot{\boldsymbol{\omega}}$. As implemented in the prediction model, the continuous-time state derivatives mapped by the solver are:

$$\ddot{\mathbf{r}} = \frac{1}{m} \sum_{i=1}^8 \mathbf{f}_i + \mathbf{g} \quad (2.3)$$

$$\dot{\boldsymbol{\omega}} = I_{world}^{-1} \left(\sum_{i=1}^8 (\mathbf{p}_i - \mathbf{r}) \times \mathbf{f}_i - \boldsymbol{\omega} \times (I_{world} \boldsymbol{\omega}) \right) \quad (2.4)$$

This formulation allows the controller to explicitly plan the necessary 3D contact forces to act on the robot’s rotational inertia, enabling active attitude recovery and robust stabilization against external disturbances that would otherwise cause a LIP-based controller to fail.

2.2 Orientation Representation Via Quaternions

Standard orientation representations, such as Euler angles, are widely used in robotics but suffer from inherent mathematical limitations. The most critical issue is the loss of one degree of freedom at certain attitudes, known as “Gimbal Lock”, which introduces mathematical singularities. In the context of a non-linear Model Predictive Control (MPC) framework, these singularities can severely destabilize numerical optimization solvers, such as the IPOPT solver used in this architecture.

To ensure a robust, singularity-free formulation, this framework describes the spatial rotation and the orientation of the robot’s single rigid body using unit quaternions. A quaternion is defined as a 4-dimensional vector comprising a real scalar component and a 3D imaginary vector part. It is commonly written as:

$$q = q_0 + q_1 i + q_2 j + q_3 k \quad (2.5)$$

The algebra of quaternions provides an elegant and computationally efficient way to map spatial rotations without relying on complex trigonometric nonlinearities. The fundamental operation is the quaternion product (denoted here by $\circ$). Given two quaternions, their product is non-commutative and is defined as:

$$q \circ p = \begin{bmatrix} p_0 q_0 - \mathbf{p} \cdot \mathbf{q} \\ p_0 \mathbf{q} + q_0 \mathbf{p} + \mathbf{q} \times \mathbf{p} \end{bmatrix} \quad (2.6)$$

Using unit quaternions, meaning that the norm of the quaternion is equal to one, spatial rotations can be elegantly mapped. A 3D vector x expressed in the fixed reference frame can be rotated into the body reference frame vector $\mathbf{x}'$ using the quaternion q and its conjugate q^* :

$$\mathbf{x}' = q \circ \mathbf{x} \circ q^* \quad (2.7)$$

In the formulation of the continuous-time Optimal Control Problem, it is crucial to link the rate of change of the spatial orientation to the physical angular velocity vector of the robot’s base frame. The time derivative of the orientation quaternion is linearly related to the angular velocity $\boldsymbol{\omega}$ through the quaternion product:

$$\dot{\mathbf{q}} = \frac{1}{2} \boldsymbol{\omega} \circ \mathbf{q} \quad (2.8)$$

Where the physical angular velocity is treated as a pure quaternion with a zero scalar part.

In our software implementation using the CasADi framework, this product is transcribed via matrix multiplication to easily compute the state-space derivatives without singularities:

$$\dot{\mathbf{q}} = \frac{1}{2} E^T \boldsymbol{\omega} \quad (2.9)$$

E^T (called E_q later) is a 4×3 matrix constructed directly from the scalar and vector components of the current orientation quaternion. This matrix acts as a mapping operator that transforms the 3D angular velocity vector $\boldsymbol{\omega}$ into the 4D quaternion derivative rate. This linear relationship allows the control architecture to seamlessly integrate the 3D spatial rotations into the prediction model. As a result, the solver can explicitly compute the required ground reaction forces to stabilize the robot’s attitude during locomotion, effectively keeping track of the orientation without ever encountering Gimbal Lock issues.

2.3 Coupling SRBD Rotational Dynamics And Quaternion Kinematics

To integrate the rotational dynamics into the prediction horizon without encountering mathematical singularities such as Gimbal Lock, the angular velocity extracted from the Euler equation must be strictly coupled with the quaternion representation. By directly feeding the dynamically computed angular velocity into the quaternion propagation kinematics, the continuous-time rotational state is fully defined. Through this explicit coupling, the MPC is able to directly map the 3D contact forces exerted at the feet to the singularity-free evolution of the robot's spatial attitude. This allows the solver to natively predict and control the base orientation over the entire horizon, laying the mathematical foundation for the control architecture implemented in the subsequent stages.

2.4 Rotational Dynamics And Inertia Tensor Projection

The rotational dynamics of the Single Rigid Body (SRB) model describe how external Ground Reaction Forces (GRF) and internal gyroscopic effects influence the humanoid base orientation. To maintain physical consistency in the World Frame, the controller must account for the time-varying nature of the inertia tensor as the robot rotates.

The robot's inertia tensor $I_{body} \in \mathbb{R}^{3 \times 3}$ is computed a priori and remains constant only within the local body-fixed frame. Since the model predictive controller operates in the inertial World Frame, the inertia tensor must be dynamically projected at each prediction step using the current base orientation. This is achieved through the rotation matrix R_q derived from the unit quaternion:

$$I_{world} = R_q I_{body} R_q^T \quad (2.10)$$

This projection ensures that the relationship between angular momentum and angular velocity remains valid regardless of the robot's spatial attitude.

The evolution of the base angular velocity ω is governed by the Euler's equations for a rotating rigid body. The angular acceleration is derived by balancing the total external torque with the change in angular momentum:

$$\dot{\omega} = I_{world}^{-1}(\tau_{tot} - \omega \times I_{world}\omega) \quad (2.11)$$

where:

$$\tau_{tot} = \sum_{i=1}^8 (\mathbf{p}_i - \mathbf{r}) \times \mathbf{f}_i \quad (2.12)$$

is the total external torque generated by the 3D contact forces f_i applied at the foot vertices p_i relative to the CoM position r . The term $\omega \times (I_{world}\omega)$ represents the non-linear gyroscopic terms arising from the base rotation, which are explicitly modeled to ensure stability during high-speed recovery maneuvers.

Chapter 3

Control Architecture And Implementation

The control architecture developed for this project is based on a hierarchical, three-layer structure. This “separation of tasks” paradigm is essential to bypass the high computational cost of computing the full-body dynamics of a humanoid robot, allowing the system to react in real-time to external disturbances and maintain dynamic stability. The architecture proceeds from high-level step generation, through the optimization of the Center of Mass (CoM) trajectories and contact forces, down to the low-level calculation of individual joint torques.

3.1 System Overview And Architecture

The developed control system is structured in a hierarchical, multi-level architecture that effectively decouples complex control tasks based on their frequency and optimization horizon. This hierarchical approach, common in modern humanoid robotics, ensures high-performance tracking and dynamic balance while maintaining the computational feasibility required for real-time control.

The overall flow of the control pipeline can be summarized as follows:

- **High-Level: Footstep Planning.** At the highest level, the Footstep Planner processes a desired velocity reference by integrating a virtual unicycle kinematic model. Leveraging this simplified representation of the global path, it iteratively generates a discrete sequence of nominal landing positions for both feet by applying a fixed lateral displacement relative to the unicycle trajectory. This sequence serves as the primary guidance for the robot’s locomotion task.
- **Mid-Level: Model Predictive Control (SRBD-MPC).** The SRBD-MPC forms the core of the dynamic control layer. Based on the Single Rigid Body Dynamics (SRBD) model, it solves an Optimal Control Problem at each control tick. It receives the nominal footstep sequence from the planner and the full robot state feedback. Its primary role is to find optimal, physically consistent Ground Reaction Forces that satisfy the friction cone and unilateral contact constraints, ensuring the robot tracks the target CoM and base trajectories.
- **Low-Level: Whole-Body Control (WBC).** The Whole-Body Controller (WBC) acts as the low-level joint-space servo, closing the control loop at a very high frequency. It receives the optimal contact forces computed by the MPC, along with full robot state feedback and desired joint-space references (e.g., arm swing for balance). Formulated as a Quadratic Programming (QP) problem, the WBC computes the exact set of joint torques that perfectly satisfy the desired high-level behaviors while adhering to strict robot constraints (e.g., joint limits, physical contact conditions).

This structure ensures that the computationally heavy optimization tasks are handled at the lower rates in the mid-level MPC, while the fast reflexes and actuator-level constraints are managed at the high-frequency WBC layer, resulting in a robust, physically consistent, and reactive dynamic walking behavior.

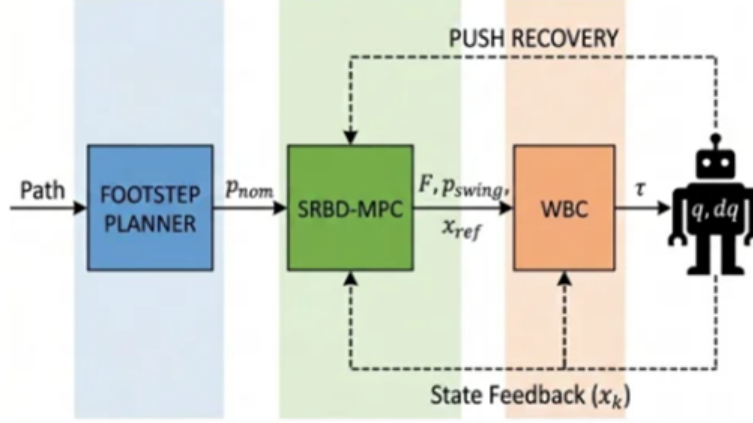


Figure 1: Layered Control Scheme

3.2 Footstep Planning And Swing Trajectories Generation

The locomotion process is initialized by defining a sequence of footprints that guide the robot toward its goal. This task is split into two specialized modules: a discrete planner that determines the landing locations and a continuous generator that computes the Cartesian trajectories for the feet during the swing phase.

3.2.1 Discrete Footstep Planning

The Footstep Planner is responsible for providing the high-level reference for the gait sequence. In this implementation, the planner generates a nominal sequence of footsteps based on a predefined reference path, specifying the Cartesian coordinates x, y, z and the orientation for each footprint.

- **Gait Timing:** The gait is organized into alternating single-support (SS) and double-support (DS) phases.
- **Reference Plan:** The planner provides the MPC with the nominal target for the next landing position P_{nom} .
- **Adaptive Behavior:** While the initial plan is static, the planner is designed to be interfaced with the MPC feedback loop. This allows the nominal targets to be updated dynamically if the Reactive Replanning logic (detailed in Chapter 5.1) detects a significant deviation due to external disturbances.

3.2.2 Continuous Swing Trajectory Generation

Once a landing target is confirmed, the Foot Trajectory Generator calculates the smooth Cartesian path that the swing foot must follow to move from the current takeoff position to the target foothold.

- **Horizontal Motion (X-Y Plane):** For the horizontal coordinates and the foot orientation, a cubic polynomial is employed. This ensures that the swing foot starts and ends its motion with zero velocity, preventing abrupt jerks that could destabilize the Single Rigid Body. The trajectory for a generic coordinate s over a swing duration T is formulated as:

$$s(t) = s_{start} + (s_{target} - s_{start}) \cdot \left(-2\frac{t^3}{T^3} + 3\frac{t^2}{T^2} \right) \quad (3.1)$$

- **Vertical Motion (Z-axis):** To ensure ground clearance and avoid accidental collisions with the terrain, the vertical profile follows a quartic polynomial. This formulation allows the foot to reach

the peak height h_{step} exactly at the midpoint of the swing phase ($T/2$) and return to the ground with zero impact velocity:

$$z(t) = 16h_{step}\frac{t^4}{T^4} - 32h_{step}\frac{t^3}{T^3} + 16h_{step}\frac{t^2}{T^2} \quad (3.2)$$

The generator provides the desired position, velocity, and acceleration for each foot at every simulation tick. These references are then fed into the Whole-Body Controller (WBC) to be mapped into joint torques.

3.3 Model Predictive Control (SRBD-MPC)

The core of the real-time approach is the Model Predictive Control based on the Single-Rigid Body Dynamics (SRBD), implemented in the `SrbdMpc` class. This layer operates at 20 Hz and optimizes contact forces over a prediction horizon of $N = 50$.

3.3.1 State And Control Variables

The implementation of the SRBD-MPC requires a formal definition of the system's state and decision variables. By abstracting the humanoid as a single rigid body, the dynamics are fully described by the motion of the Center of Mass (CoM) and the base orientation.

The State Vector: To ensure a singularity-free representation of the robot's attitude, the orientation is expressed through unit quaternions. The state vector $x \in \mathbb{R}^{13}$ is defined as:

$$\mathbf{x} = [\mathbf{p}^T, \dot{\mathbf{p}}^T, \mathbf{q}^T, \boldsymbol{\omega}^T]^T \quad (3.3)$$

Where:

- $\mathbf{p} \in \mathbb{R}^3$ is the position of the CoM in the world frame.
- $\dot{\mathbf{p}} \in \mathbb{R}^3$ is the linear velocity of CoM.
- $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$ is the unit quaternion representing the base orientation.
- $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ is the angular velocity in world frame.

To integrate the angular velocity $\boldsymbol{\omega}$ into the dynamics of the quaternion $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$, the mapping operator $E^T = E_q$ is constructed in the code by exploiting the linearity of the quaternion product:

$$E_q = \begin{bmatrix} -q_x & -q_y & -q_z \\ q_w & q_z & -q_y \\ -q_z & q_w & q_x \\ q_y & -q_x & q_w \end{bmatrix} \quad (3.4)$$

Furthermore, to transform quantities from the local frame to the global frame (World Frame), the framework extracts the rotation matrix R_q directly from the quaternion state via the formula implemented in the `quat_to_rot` method:

$$R_q = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_z q_w) & 2(q_x q_z + q_y q_w) \\ 2(q_x q_y + q_z q_w) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_x q_w) \\ 2(q_x q_z - q_y q_w) & 2(q_y q_z + q_x q_w) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix} \quad (3.5)$$

As regards the control inputs, to effectively generate control moments and ensure physical controllability (especially during the single-support phase), surface contacts cannot be approximated as a single point. In the proposed CasADi MPC implementation, the contact surfaces are modeled using $N_v = 4$ corner vertices for each of the $N_c = 2$ feet. This discretization yields a total of 8 active contact points $c_{i,j}$, where $i \in \{L, R\}$ and $j \in \{1, 2, 3, 4\}$.

Since each vertex can exert a tridimensional Ground Reaction Force (GRF) vector $f_{i,j}$, the control input u of the continuous state-space model is a 24-dimensional vector:

$$\mathbf{u} = [\mathbf{f}_{L,1}^T \dots \mathbf{f}_{L,4}^T \quad \mathbf{f}_{R,1}^T \dots \mathbf{f}_{R,4}^T]^T \in \mathbb{R}^{24} \quad (3.6)$$

This formulation is critical for two reasons:

1. **Torque Distribution:** It allows the MPC to naturally generate ankle torques by differently weighting the vertical forces among the vertices of the same foot.
2. **Center of Pressure Control:** By optimizing the 24 force components, the solver implicitly constrains the CoP to remain within the physical boundaries of the rectangular footprint, ensuring that the robot does not tip over even when subjected to external lateral disturbances.

3.3.2 Discretization And Prediction Horizon

In this implementation, we employ the Direct Multiple Shooting (DMS) method. The DMS approach discretizes the horizon T into N shooting nodes. At each node k , both the state x_k and the control input u_k are treated as decision variables.

The temporal parameters of the MPC are set as follows:

- Prediction Horizon N : 50 nodes.
- Sampling Time Δt : 0.01 seconds.
- Total Look-ahead Time: $T_{hor} = N \cdot \Delta t = 0.5$ s.

The continuous dynamics $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ are integrated between nodes using an Euler integration scheme, resulting in the discrete-time equality constraints:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + f(\mathbf{x}_k, \mathbf{u}_k)\Delta t \quad (3.7)$$

Furthermore, the Receding Horizon principle is applied: at each control loop iteration, only the first optimal control input u_0^* is sent to the Whole-Body Controller, and the entire optimization is re-initialized at the next time step with the updated robot state feedback.

3.3.3 Cost Function Formulation

The Model Predictive Control (MPC) computes the optimal GRFs and Center of Mass (CoM) trajectories by minimizing a quadratic objective function J . The function is composed of a running stage cost evaluated over the prediction horizon N , and discrete penalty terms governing the step placement. The total cost is formulated as:

$$\min_{\mathbf{X}, \mathbf{U}} \left\{ \sum_{k=0}^{N-1} (\Psi_{CoM,k} + \Psi_{quat,k} + \Psi_{reg,k}) + \Psi_{swing} + \Psi_{clearance} \right\} \quad (3.8)$$

CoM Tracking: To ensure vertical stability and prevent the robot from collapsing under gravity, the total CoM cost is defined as $\Psi_{CoM} = \Psi_{CoM,z} + \Psi_c$. The height tracking cost is formulated as:

$$\Psi_{CoM,z} = W_{com,z} |z_k - h_{ref}|^2 \quad (3.9)$$

The horizontal progression of the CoM is guided by the dynamically updated contact sequence. To prioritize forward momentum while allowing lateral compliance, an anisotropic weight matrix $W_{com,xy} = \text{diag}(W_{com,x}, W_{com,y})$ is employed:

$$\Psi_c = (\mathbf{p}_{xy,k} - \mathbf{p}_{xy,target})^T W_{com,xy} (\mathbf{p}_{xy,k} - \mathbf{p}_{xy,target}) \quad (3.10)$$

Swing Cost: A quadratic penalty term is added to softly anchor the foot to the nominal trajectory calculated by the footstep planner:

$$\Psi_{swing} = W_{swing} \|\mathbf{p}_{swing} - \mathbf{p}_{target}\|^2 \quad (3.11)$$

Lateral Clearance Penalty: To prevent leg cross-over and self-collisions during severe impact recoveries, a soft quadratic penalty is applied to the objective function if the swing foot violates the minimum lateral safety margin $d_{min} = 0.115$ m with respect to the support foot, which corresponds to the nominal foot-hip lateral distance in the HRP-4. To correctly penalize cross-overs in any direction, the penalty uses a signed directional check based on the support foot side:

$$\Psi_{clearance} = \begin{cases} W_{lat} \max(0, p_y^{swing} - (p_y^{support} - d_{min}))^2 & \text{if left foot in support} \\ W_{lat} \max(0, (p_y^{support} + d_{min}) - p_y^{swing})^2 & \text{if right foot in support} \end{cases} \quad (3.12)$$

Orientation And Quaternion Integrity: The attitude control independently penalizes the spatial orientation errors. Given the desired yaw reference q_{ref} , the quaternion error components $\mathbf{q}_{err}$ are explicitly computed via quaternion multiplication and minimized alongside a unit-norm regularization:

$$\Psi_{quat} = W_{q,roll}q_{err,x}^2 + W_{q,pitch}q_{err,y}^2 + W_{q,yaw}q_{err,z}^2 + W_{norm}(\|\mathbf{q}_k\|^2 - 1.0)^2 \quad (3.13)$$

Regularization And Control Effort: This term penalizes excessive forces and ensures motion smoothness through quadratic penalization of CoM velocity, base angular velocity, and control inputs:

$$\Psi_{reg} = \dot{\mathbf{p}}_k^T W_v \dot{\mathbf{p}}_k + \boldsymbol{\omega}_k^T W_\omega \boldsymbol{\omega}_k + \mathbf{u}_k^T W_f \mathbf{u}_k \quad (3.14)$$

3.3.4 Physical And Kinematic Constraints

To guarantee that the optimal control trajectories generated by the NLP solver are physically implementable and safe for the HRP-4 humanoid platform, the optimization problem is subject to a set of hard constraints. These constraints represent the physical boundaries of the system, including dynamic consistency, contact stability, and kinematic limits.

Dynamic Consistency Constraints: The state evolution must strictly adhere to the discretized nonlinear dynamics of the Single Rigid Body model. In the Direct Multiple Shooting formulation, this is enforced through equality constraints at each shooting node k :

$$\mathbf{x}_{k+1} = \mathbf{x}_k + f(\mathbf{x}_k, \mathbf{u}_k)\Delta t, \quad \forall k \in \{0, \dots, N-1\} \quad (3.15)$$

This ensures that the predicted state trajectory is time-consistent and physically reachable given the optimized control inputs $\mathbf{u}$.

Dynamic Continuity Constraints: In a Multiple Shooting approach, the state at each node k is treated as an independent decision variable. Therefore, an additional continuity condition (or defect constraint) is required to “link” the independent shooting intervals:

$$\mathbf{x}_{k+1} - \hat{\mathbf{x}}_{k+1} = 0 \quad (3.16)$$

Where $\hat{\mathbf{x}}_{k+1}$ represents the end-state resulting from the integration of the dynamics from the previous node. Setting this “defect” to zero guarantees that the separate segments join perfectly at the boundaries, resulting in a single, seamless trajectory.

Unilateral Contact And Friction Cone Constraints: To maintain a non-slip contact between the robot’s feet and the terrain, the Ground Reaction Forces (GRF) must reside within the friction cone. To maintain linearity and reduce computational complexity, a pyramidal approximation is employed:

$$|f_{x,i,j}| \leq \mu f_{z,i,j}, \quad |f_{y,i,j}| \leq \mu f_{z,i,j} \quad (3.17)$$

Furthermore, the unilateral nature of the contact (i.e., the ground can only push the robot) is enforced by constraining the vertical force components to be strictly non-negative and bounded to prevent structural saturation:

$$0 \leq f_{z,i,j} \leq f_{max} \quad (3.18)$$

where f_{max} is set to 500 N to ensure a safe distribution of the robot’s weight across the active contact vertices.

Complementarity And Swing Phase Constraints: The gait sequence dictates which contact points are active. During the swing phase of a foot, the corresponding GRF vectors are strictly forced to zero to avoid unrealistic “internal forces” generated by the optimizer:

$$\mathbf{u}_{swing,k} = \mathbf{0} \quad (3.19)$$

Total Vertical Force Limits: To prevent unrealistic jump-like transients and ensure stable load distribution, the sum of all vertical forces is tightly bounded around the robot’s weight mg . The constraints dynamically adjust based on the current gait phase (Single or Double Support):

$$\gamma_{min}mg \leq \sum_{i=1}^8 f_{z,i,k} \leq \gamma_{max}mg \quad (3.20)$$

where γ_{min} and γ_{max} are scaling boundaries strictly tuned for the SS and DS phases.

Angular Velocity Bounds: To prevent numerical explosions within the non-linear rotational dynamics of the solver, strict hard bounds are imposed on the predicted base angular velocity ω_k :

$$\omega_{min} \leq \omega_k \leq \omega_{max} \quad (3.21)$$

where the limits are empirically set to ± 8 rad/s for roll and pitch rates, and ± 4 rad/s for the yaw rate.

Post-Optimization Kinematic Sanitization: While the MPC solver provides optimal footstep targets, a downstream safety filtering layer explicitly enforces a minimum lateral separation of 0.16 m between the swing foot target and the support foot center. If a required recovery step threatens this clearance, the controller dynamically truncates the longitudinal step length to maintain the gap, ensuring the final command remains strictly within the robot’s 0.35 m maximum radial kinematic workspace.

Chapter 4

Whole-Body Control (WBC) And Inverse Dynamics

The final link in the control chain, executed at a frequency of 100 Hz matching the simulator's time step, aims to translate the Cartesian poses and optimal forces computed by the MPC into the torques to be delivered to the individual motors τ . This operation is handled by the `InverseDynamics` class, which formulates a Quadratic Programming (QP) problem.

The pure contact forces (ideal points in the MPC) undergo a transformation to calculate the real torques. The forces calculated at the four corners of each foot are summed, and their cross products $\tau = \mathbf{r} \times \mathbf{f}$ generate the rotational moment, converting them into 6D wrenches expressed in the local frame of the soles. The framework iteratively extracts the Jacobian Matrices (J) and their time derivatives ($\dot{J}$) for all critical tasks (feet, CoM, torso, base, and redundant joints).

4.1 QP Problem Formulation

The WBC searches for the vector of optimal joint accelerations $\ddot{q}$, joint torques τ , and actual contact forces f_c that minimize a weighted sum of task space tracking errors, reference force tracking, and control effort regularization. The quadratic objective follows the form:

$$\min_{\ddot{q}, f_c, \tau} \left\{ \sum_{k \in \text{tasks}} W_k \left\| J_k \ddot{q} + \dot{J}_k \dot{q} - \ddot{x}_{des,k} \right\|^2 + W_{f_c} \|f_c - f_{c,ref}\|^2 + W_\tau \|\tau\|^2 \right\} \quad (4.1)$$

Where $\ddot{x}_{des,k}$ includes the feedforward terms provided by the MPC, added to the feedback tracking errors on position and velocity (via the gains `pos_gains` and `vel_gains`). For each controlled task (CoM, base, torso, swinging feet), the desired Cartesian space acceleration $\ddot{x}_{des}$ provided to the Quadratic Programming solver is not simply the optimized target, but the output of a closed-loop controller. The control law fuses the feedforward term $\ddot{x}_{ff}$ generated by the MPC/Planner with the correction terms for the position error $e = x_{des} - x_{cur}$ and velocity error $\dot{e} = \dot{x}_{des} - \dot{x}_{cur}$ measured by the simulator:

$$\ddot{x}_{des} = \ddot{x}_{ff} + K_d(\dot{x}_{des} - \dot{x}_{cur}) + K_p(x_{des} - x_{cur}) \quad (4.2)$$

The gain matrices K_p and K_d , as well as the task priority weights W_k , have been rigorously calibrated for each operational task within the `get_joint_torques` method to prioritize the tracking of the center of mass and the base orientation.

The minimizer (OSQP solver) is subjected to rigorous equality and inequality constraints:

- **Dynamic Constraint (Equality):** The full-body equation of motion is imposed:

$$M(q)\ddot{q} + h(q, \dot{q}) = S^T \tau + J_c^T f_c \quad (4.3)$$

(The mass matrix M , Coriolis and gravity forces combined with the selection matrix S and the contact Jacobian J_c , ensure that the generated torques produce physically correct accelerations).

- **Contact Constraints (Inequality):** The friction cones are rigorously re-imposed for the 6D components, including the Center of Pressure (CoP) limits, which force the reactions to reside exclusively within the geometric footprint of the foot $-\frac{d}{2} \leq \text{CoP}_{x,y} \leq \frac{d}{2}$, along with the unilateral ground pushing constraint $f_z \geq 0$.

- **Hard Joint Limits:** Constraints to prevent hardware self-damage by bounding the maximum angular excursions (particularly of the neck and chest). This is achieved by directly limiting the acceleration imposed on the joint in the next integration step:

$$\ddot{\mathbf{q}}_{min} \leq \ddot{\mathbf{q}} \leq \ddot{\mathbf{q}}_{max} \quad (4.4)$$

The QP solver solution extracts the torques for all 28 actuated degrees of freedom of the robot. As a final safeguard measure for both the simulation and the real-world counterpart, the output effort commands are saturated according to the specific motor capacitive limits provided by the robot’s URDF model (falling back to a generic ± 100 Nm only if the model limits are invalid or undefined) before being sent to the simulation layer in DART.

4.2 Knee Anti-Hyperextension Guard

In addition to pure optimization, the framework integrates a low-level safety logic, which is fundamental for the real-world counterpart, which is the Knee Anti-Hyperextension Guard. To prevent kinematic singularities and structural damages typical of humanoid robots, the WBC employs a dual-layer protection scheme. First, it dynamically imposes strict inequalities on the angular acceleration of the knees within the QP constraints. Secondly, by proactively calculating the future pose $\mathbf{q}_{pred} = \mathbf{q}_{now} + \Delta t \cdot \dot{\mathbf{q}}_{now}$ as a post-optimization step, the algorithm injects a PD-based corrective torque if the joint dangerously approaches the safety limit, ensuring a safe flexion margin cleanly overrides the optimal command.

Chapter 5

Advanced Disturbance Recovery Strategies

5.1 Reactive Footstep Replanning (MPC Level)

The most effective strategy for rejecting high-energy lateral impacts is the dynamic relocation of footsteps.

- **Optimization-based Replanning:** Instead of using a fixed nominal target, the optimization variable P_{swing} is allowed to deviate from the center of the footprint.
- **Two-layered gait recovery:** The solver will naturally try to use first the available DoFs to absorb the impact and not deviate the robot from its initial plan (e.g. twisting itself). When this is not possible, a new footstep is recomputed dynamically to slow down the lateral movements induced by the pushing forces. The MPC then recomputes also the variation in angular momentum and the needed contact forces.

5.2 Dynamic Cost Weight Scheduling (Replanning Mode)

Instead of maintaining a strictly constant objective function throughout the entire gait cycle, the MPC framework implements a dynamic weight scheduling strategy to handle severe impacts. When a major external disturbance is detected and the reactive footstep replanning is triggered (as described in Chapter 5.1), the controller simultaneously enters a "Replanning Mode". In this state, the optimization weights governing the upper-body attitude tracking, specifically the quaternion orientation error W_{quat} and the base angular velocity regularization W_{ω} , are dynamically relaxed. For example, during a high-energy lateral impact, the penalty on the torso's roll and yaw tracking errors, as well as the angular velocity penalty, are significantly reduced. This strategic relaxation effectively transforms the robot's upper body into a more mathematically compliant system. Rather than rigidly resisting the impact, which could lead to control effort saturation, friction cone violations, or IPOPT solver failures, the MPC is allowed to safely "yield". By granting the solver more flexibility on the spatial orientation, the robot can absorb the kinetic energy of the disturbance by temporarily rotating its torso. Concurrently, the weight for the Center of Mass height tracking ($W_{CoM,z}$) is maintained at a strictly high value (e.g. 5000.0) to ensure vertical stability and prevent the robot from collapsing or kneeling excessively. Once the recovery step is successfully executed, the support polygon is re-established, and the unmodeled momentum is dissipated, the MPC smoothly restores the weights to their nominal, rigid values, allowing the robot to converge back to a highly structured and upright walking posture.

5.3 Kinematic Momentum Control (Dynamic Arm Swing)

Bipedal locomotion naturally generates unwanted yaw moments (L_z) around the vertical axis due to the forward and backward acceleration of the swing and support legs. To maintain balance without introducing the massive computational burden of full whole-body momentum optimization, an active, bio-inspired arm swinging strategy is implemented.

In the main simulation loop, a proportional kinematic coupling logic has been introduced. The shoulder pitch joints are dynamically coupled to the longitudinal extension of the legs relative to the Center of Mass. This heuristic ensures that the arms move in opposition to the legs, effectively generating a counter-moment. The tracking reference for the arm is calculated as:

$$\theta_{arm} = -k_{swing}\Delta p_{leg} \quad (5.1)$$

Where θ_{arm} is the commanded shoulder pitch angle, Δp_{leg} represents the leg’s longitudinal displacement along the X-axis relative to the CoM, and $k_{swing} = 1.5$ is an empirical gain factor tuned to maximize yaw stability. This computationally lightweight approach significantly improves the heading tracking without burdening the NLP solver.

5.4 Numerical Reliability: Safe Fallback And Warm-Starting

To guarantee real-time convergence of the IPOPT solver within the required latency threshold, a Warm-Start strategy is implemented. Since the MPC operates at 20 Hz and the WBC/Simulation at 100 Hz, the solver initializes the current prediction horizon by shifting the optimal state and control trajectories from the previous iteration by a 5-tick offset:

$$\mathbf{X}_{guess} = [\mathbf{x}_{shift}^*, \mathbf{x}_{shift+1}^*, \dots, \mathbf{x}_N^*, \mathbf{x}_N^*, \dots] \quad (5.2)$$

This technique allows the non-linear solver to converge consistently in approximately 55 ms, well below the 75 ms hard limit required for real-time viability.

In the rare event of NLP non-convergence, the framework triggers a graceful degradation mode that uniformly distributes the robot’s weight across the active contact vertices, preventing immediate collapse while the next optimization cycle is initialized.

Chapter 6

Experiments And Results Analysis

6.1 Testing Environment, Simulation Setup And Robot Physical Parameters

The control algorithm was tested on a HRP4 humanoid robot inside the aforementioned DARTpy physics simulations framework. The HRP4 is a humanoid robot designed and built by Kawada Industries and Japan's National Institute of Advanced Industrial Science and Technology (AIST) in 2010, and its specifications are specified in the table below:

Table 1: HRP-4 relevant specifications

Specification	Value
Height	151 cm
Weight	40 kg
Degrees of Freedom (DOFs)	34 total: 6 DOFs per leg, 7 DOFs per arm, 2 DOFs per hand, 2 DOFs in the neck, 2 DOFs in the chest
Actuators	80W + low-power motors
Materials	Plastic (cover), Aluminum alloy (frame)

The tests were run on a Desktop PC mounting a Ryzen 7 5800X 8-Core processor @ 4.20 GHz, an Nvidia RTX 3070 8 GB graphics card, and 16 GB of DDR4 3600 MHz RAM.

To test the algorithm effectiveness, the robot has been tasked to complete a straight walking trajectory while withstanding lateral impacts of different intensities and in different walking phases, inputting exactly 1 disturbance force per test run lasting a set amount of time. Both the tracking capability and robustness is tested, as the initial trajectory that the robot should follow is output by the footstep planner at the beginning of the simulation, but the MPC should be able to replan accordingly reacting to the disturbances, and to also make the robot recover eventual drifts from the initial trajectory that it may be inflicted with from trying not to fall.

All 3 simulations last 16.0 s of simulation time, and the simulation clock ticks only when the simulation is not frozen due to high computational load, as the IPOPT solver takes up to 3-4 s per call to solve the associated OCP, thus forcing the simulation to momentarily stop to process these computations while keeping the execution in real-time.

Termination occurs when:

1. The robot falls, i.e. $z_{com} < 0.35$ m;
2. The robot completes the plan without any early termination.

Here we report the 3 most significant tests:

1. The robot walks normally with no disturbances. Used to assess basic algorithm functionality.
2. The robot gets hit with a force of 50 N to the left, with the force vector orthogonal with the sagittal axis of the body, for a total of 0.15 s, resulting in an impact of impulse 7.5 N·s, during its Double Support phase.

3. The robot gets hit in the same way with the same impulse but while in Single Support phase (right foot swinging, leftward impact).

Also we attempted to ablate the maximum absorbable impulse on a single strike, sweeping across a range of intensities for the lateral push, as well as weights for the cost function to ablate the tradeoff between compliance and energy-saving. The footstep map is shown in Fig. 3-A, 4-A, 5-A. In the next sections the disturbance rejection and the orientation and gait tracking stability will be addressed, as well as the maximum resistance.

6.2 Disturbance Rejection And Real-Time Trajectory Readaptation Vs Baseline Behavior

In the first baseline run the robot completed the task in the allotted time successfully without deviating from the starting plan and/or triggering the replanning algorithm, as it can be seen in the footstep map in Fig. 3-A.

In Fig. 3-B, 3-C and 6-top, the detected Ground Reaction Forces (GRFs) on each contact point and the torso orientation and CoM tracking information reveals how the algorithm makes the robot walk, and to let it follow the planned gait the torso orientation chases a target which has a constantly rippling yaw, and that's because the target yaw for the torso is footstep orientation-based, since the feet are instructed to chase the virtual unicycle that is used by the footstep planner to generate the initial trajectory, and by keeping in mind that this system should be ready to face disturbances at any moment it needs a temporal-coherent yaw reference to track. Naturally this target is subject to change once the replanning algorithm triggers, but it doesn't change its nature.

The GRFs graph shows also that at every instant the robot never breaks any physical constraint, by transferring all its weight to the support leg during single-support phases, and thus all the GRFs are detected only on that leg which, in any case, always amount to $40 \text{ kg} \times 9.81 \text{ m/s}^2 \cong 400 \text{ N}$ total across all 8 contact points, including the flying foot.

Now that the baseline has been established, disturbances can be introduced. First, the easy case has been explored, which is the case of a single lateral impact during a Double Support phase, which is the phase with the highest stability among all, since having both feet on the ground ensures a large enough support polygon and an even distribution of GRFs across all contact forces, such that the moment variation $L_{x,y}$ induced from non-disturbance source is minimal.

An impact on a 40 kg body with an impulse of 7.5 N-s forces a lateral shift velocity of 0.19 m/s, which is considered a strong deviation. Nonetheless, the robot manages to keep walking without losing balance by changing momentarily its footstep plan (Fig. 4-A) and absorbing the moments on the torso by readjusting the yaw without breaking actuation limits (Fig. 6-bottom). Here too all the physical limits are not broken and the GRFs also show no anomalies (Fig. 4-B, 4-C).

The tracking recovery term (Chapter 5.3) allows the robot to flawlessly return on its original plan and is also critical for disturbance rejection, since we tested other recovery options (Nothing, Drift propagation...) but they all caused IPOPT failures after the impact and caused the HRP-4 to fall 1-2 steps later.

In the third test the algorithm could also make the robot exceptionally stable under disturbance and the MPC allowed it to complete the plan in the allotted time, again with no anomalies on GRFs, actuation or tracking of CoM position/torso orientation (Fig. 5-A, 5-B, 5-C, 7).

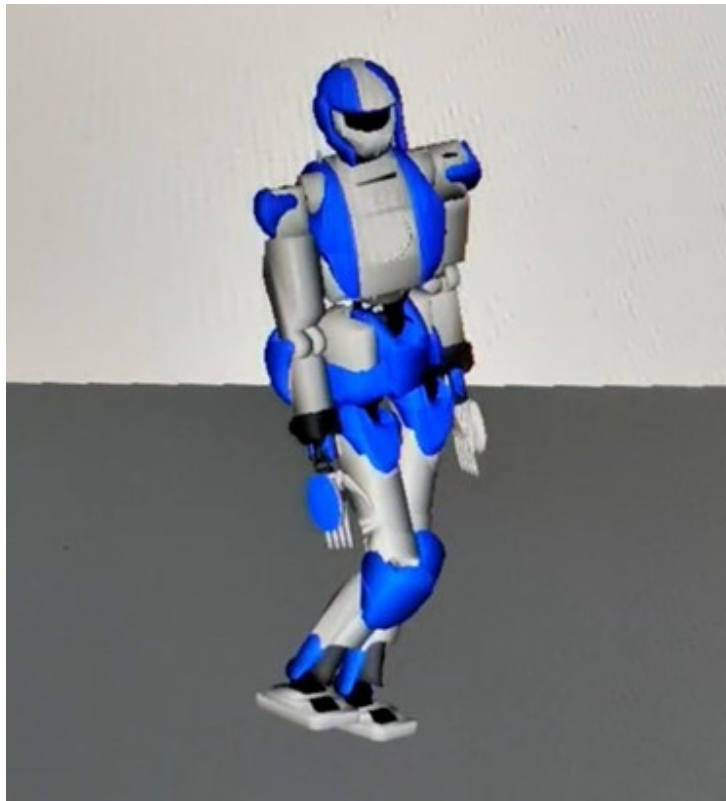


Figure 2: The robot behavior under disturbance

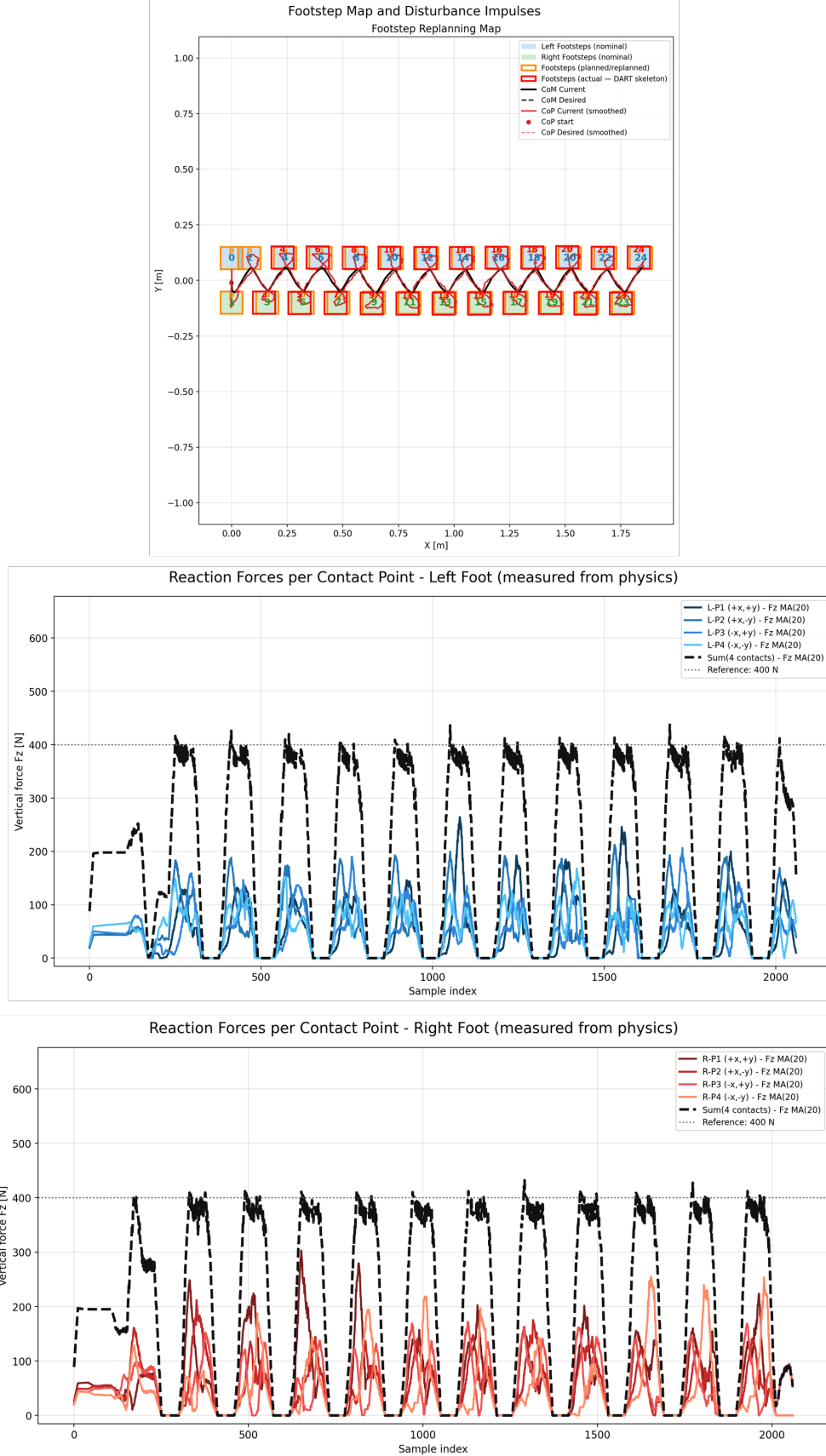


Figure 3: No Disturbance case: (A) On the top the footstep map. In the middle, (B) the left and (C) right foot GRFs for each contact point and their moving average over 20 ticks.

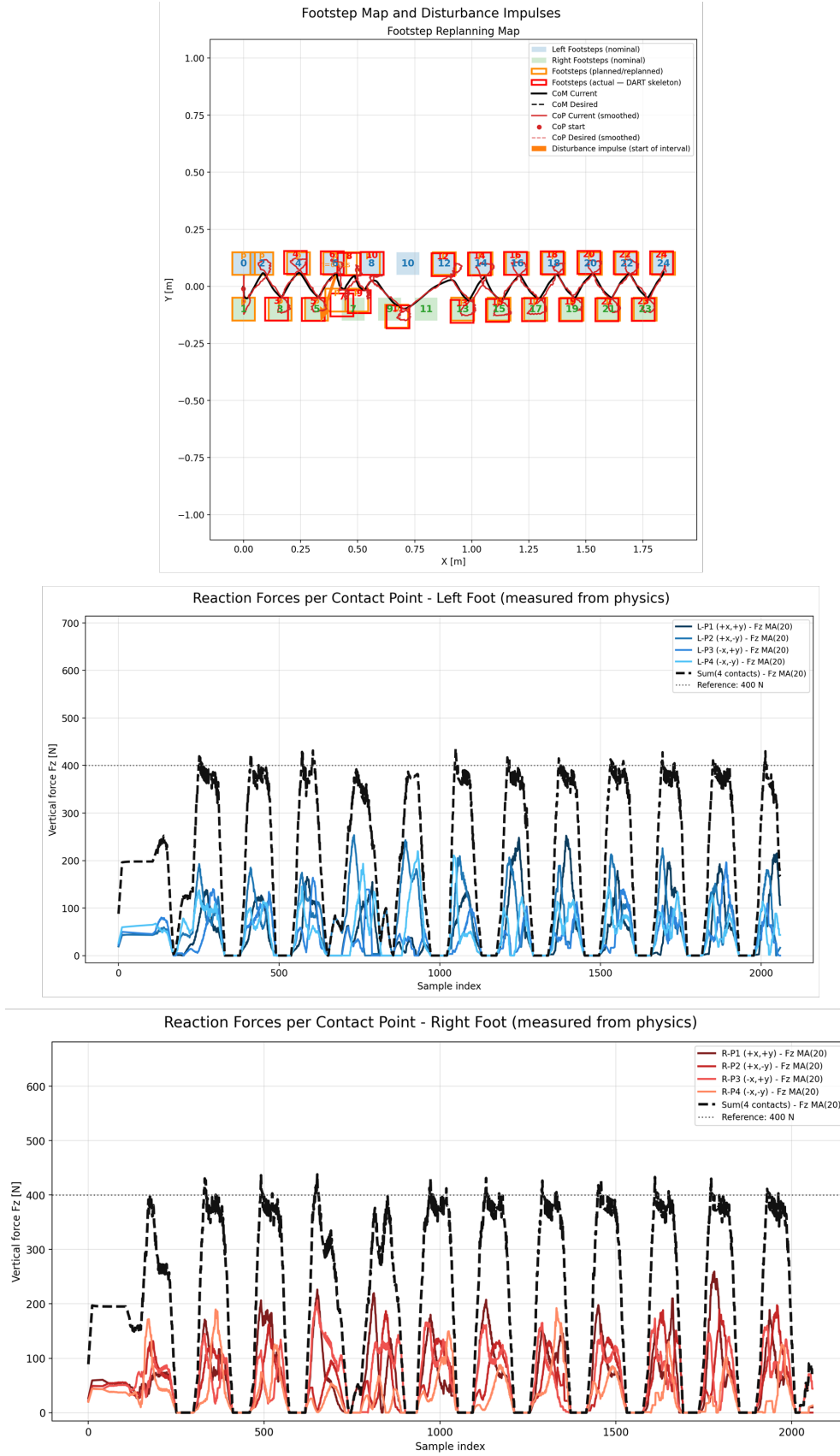


Figure 4: 50N force during DS case: (A) On the top the footstep map. In the middle, (B) the left and (C) right foot GRFs for each contact point and their moving average over 20 ticks.

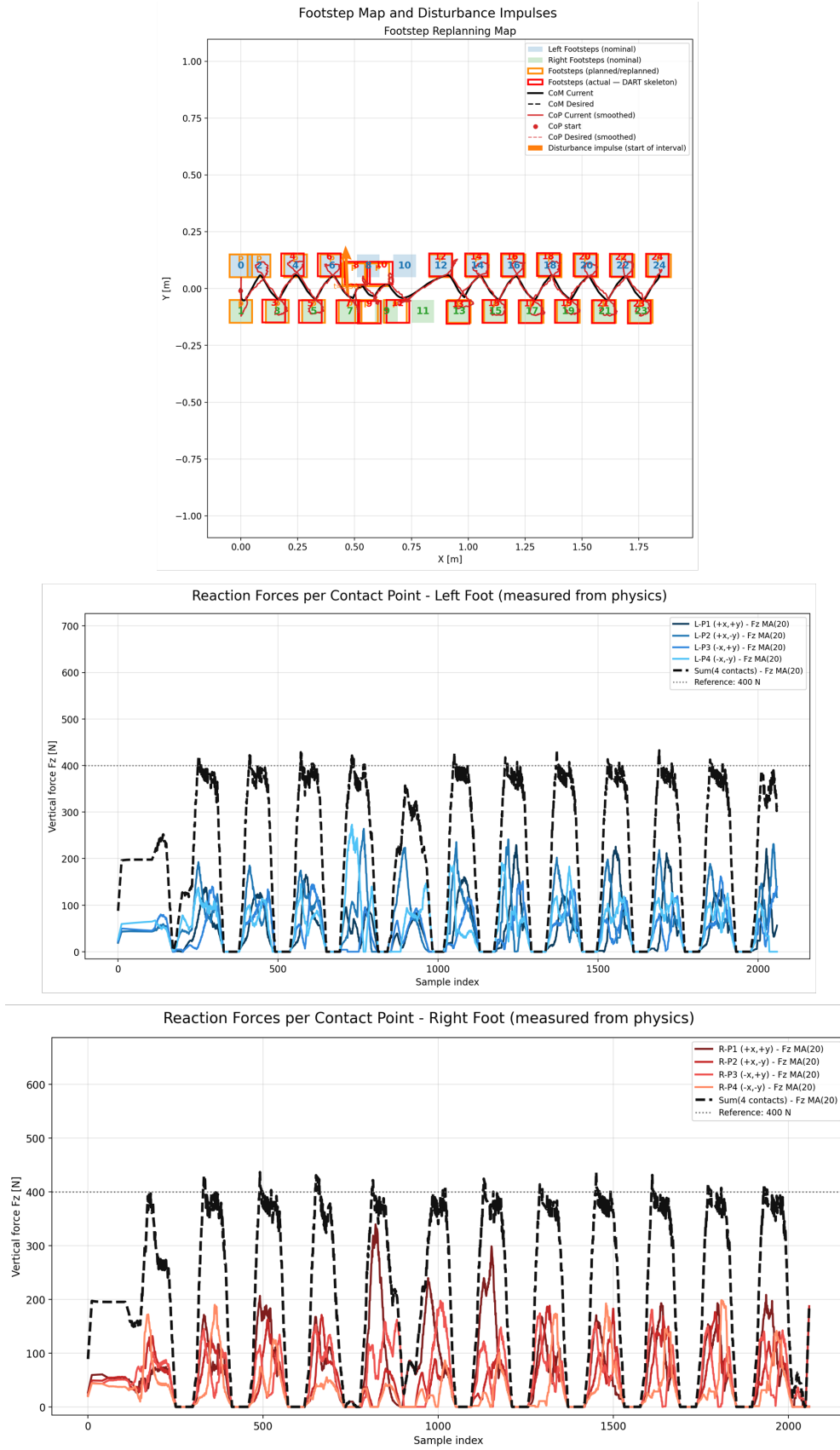


Figure 5: 50N force during SS case: (A) On the top the footstep map. In the middle, (B) the left and (C) right foot GRFs for each contact point and their moving average over 20 ticks.

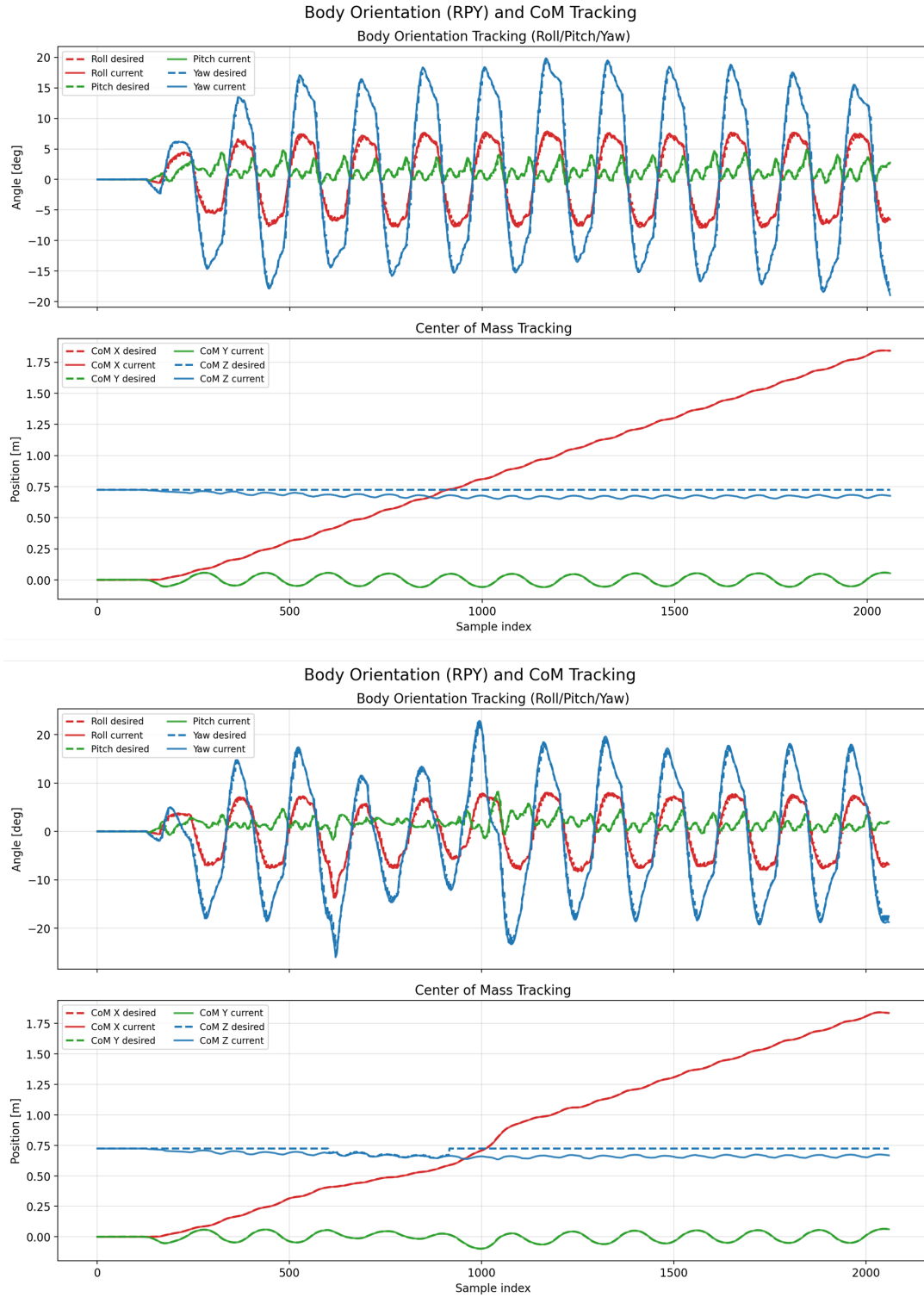


Figure 6: CoM position and Body Orientation during no disturbance test (top) and during 50N force in DS test (bottom).

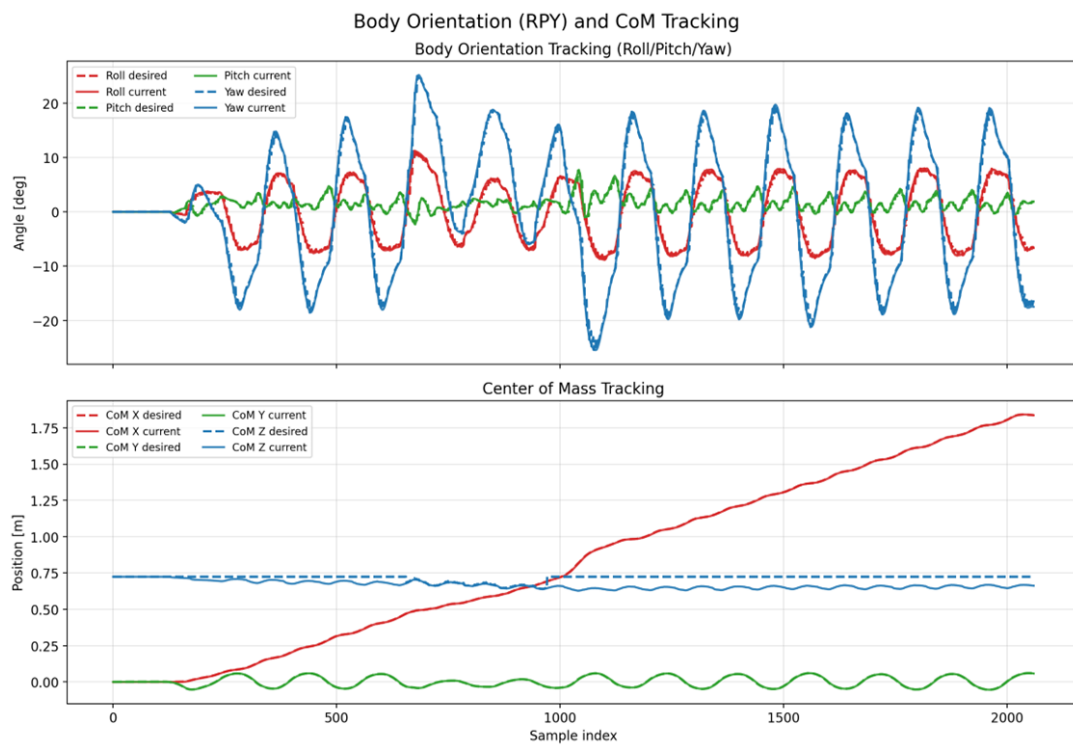


Figure 7: CoM position and Body Orientation during 50N force in SS test.

6.3 Ablation Of Impact Intensity And Compliance

Task-prioritization via weights on the cost functions can help defining and refining the desired robot behavior while accomplishing the main task, but it can also highlight limitations that can occur when dealing with multiple tasks, i.e. keeping a more natural-looking and rigid walking posture while also being able to resist impacts forces a trade-off on the controller between the two tasks, and this is demonstrated with the results of the experiments we set up here. We use 3 different setups for the robot:

1. **Compliant:** $W_{quat} = \text{diag}([15.0, 15.0, 100.0])$, $W_{omega} = \text{diag}([3.0, 3.0, 3.0])$, CoM height target less rigid.
2. **Switching:** $W_{quat} = \text{diag}([200.0, 200.0, 200.0])$, $W_{omega} = \text{diag}([10.0, 10.0, 10.0])$ during normal walking, $W_{quat} = \text{diag}([200.0, 200.0, 20.0])$, $W_{omega} = \text{diag}([10.0, 10.0, 5.0])$ during replanning, $W_{CoM,z} = 5000.0$ kept constant throughout to enforce the CoM height and avoid kneeling.
3. **Rigid:** Like the Switching profile, but only with the rigid part.

Then we test each setup by equipping it to the robot and making it perform several simulation rounds by applying an increasingly stronger disturbance in force at the same time instant at each round, always during a Double-Support phase. The results of the tests are shown in Tab. 2 below:

Table 2: Disturbance performance breakdown			
Intensity	Compliant	Switching	Rigid
≤ 50 N	✓	✓	✓
70 N	✓	✓	✗
80 N	✓	✗	✗
90 N	✗	✗	✗

These results show that by allowing the joints of the robot to turn more freely to adjust to the sudden change in angular momentum one can achieve better disturbance recovery, but it comes with a cost. As shown in Fig. 8, a more compliant robot also tends to be more unstable even when it's not disturbed, i.e. its roll angle swings between -15° and 15° , meaning that the walking posture is undulating and forces an uneven distribution of reaction forces on the contact points, also making the robot gradually kneel down (which is a natural reaction by the MPC to ensure a lower CoM and a more stable equilibrium) and causing the actuators on the joints overall to exert more torque, thus increasing the robot's energy consumption.

An entirely rigid setup solves the energy consumption issue by forcing the robot into a more erect and less undulating walking posture, but it cannot make use of the joints' shock absorption capabilities as they are not allowed to rotate as freely as they could while the disturbance force was applied.

The solution is to employ a hybrid weighting system which switches weights for the MPC between two different setups via the same replanning trigger used in Chapter 5.2. All the results produced in the previous experiments were obtained with this setup, and with more tuning a better optimum between disturbance rejection and natural-looking walk patterns/rigidity can be found. As of now, the robot after disturbance with this setup can easily trace back its steps to the original plan and distributes the reaction forces more evenly among its contact points, and the undulating pattern is greatly reduced (between -7° and 7°).

Another interesting feature of our algorithm is how, by modifying weights, the robot falling behavior can be tuned to minimize damage after failure. In fact, the more compliant the robot is (combined with the Anti-Hyperextension guard for the knees), the more the robot tends to fall safely to its knees when it gets hit with a force stronger than what it can resist. If the weights are too severe then the robot falls rigidly to the side instead of letting the legs act as shock absorbers, causing more damage to the components upon impact.

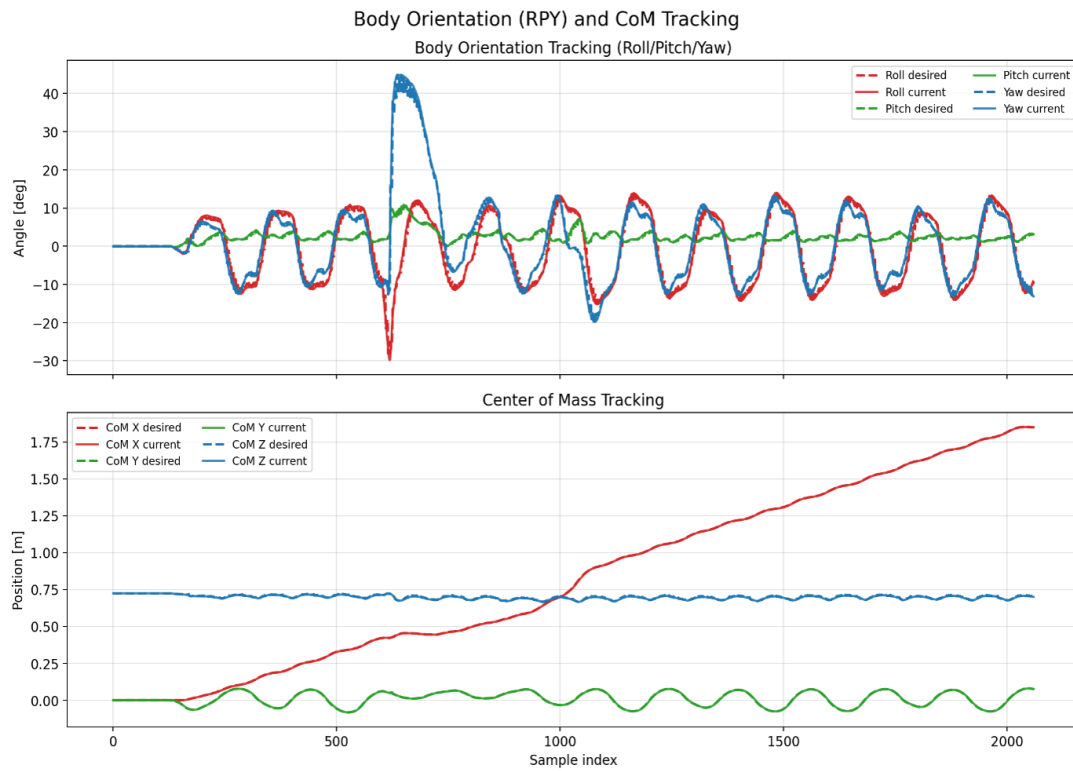


Figure 8: CoM position and body orientation tracking under heavy disturbance.

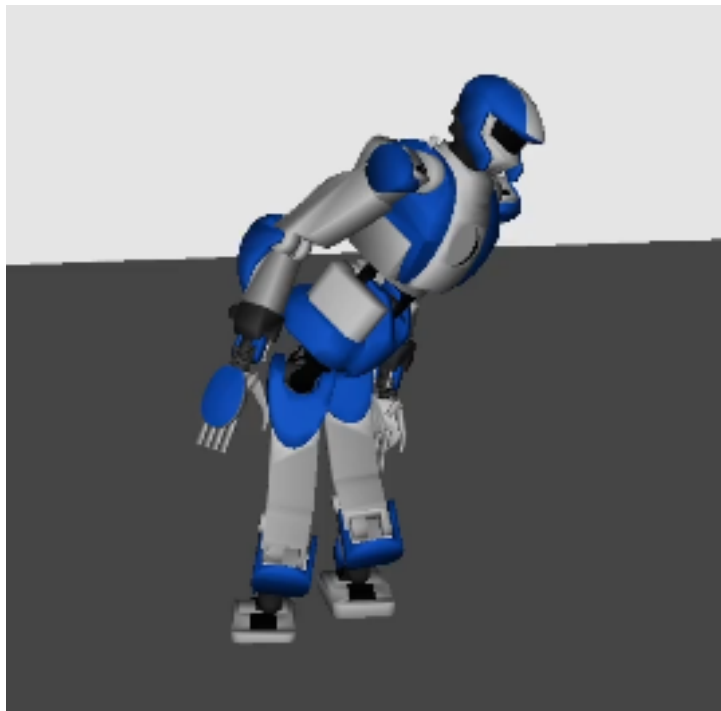


Figure 9: Robot falls after sustaining a 90 N impact, compliant setup.

Chapter 7

Conclusions And Future Work

The aim of this project is to implement an MPC on a humanoid robot, the HRP-4, using Single Rigid Body Dynamics to make it resistant to lateral impacts while completing the assigned task(s). In this case it consisted in walking on a straight line, but it could be extended to more complex trajectories since the recovery system is modeled in the MPC.

The algorithm should be able to make the robot withstand impacts of different intensities and in different phases, and it excels in doing so even with very strong impacts, avoiding catastrophic joint overloads and preserving them as much as possible. The algorithm also manages to retrace the HRP-4 steps back to the originally planned trajectory, ensuring both tracking stability and disturbance rejection.

Further developments would include the addition of sensors to detect obstacles and include an obstacle avoidance part to the algorithm (like the authors of [3] have already done), which adds a “reasoning” layer to the reactive layer (the disturbance recovery layer) we already have developed which would allow the robot to be able to complete tasks in crowded areas, with other people walking in the same operational area of the robot, and then expand the usage of this algorithm in cooperative/competitive work environments with other robots via Reinforcement Learning but still basing the computations on this SRBD-MPC + eventual Think-React algorithm which won’t rely on data-driven models.

Chapter 8

References

- [1] N. Scianca, D. De Simone, L. Lanari, and G. Oriolo, “MPC for Humanoid Gait Generation: Stability and Feasibility,” *IEEE Transactions on Robotics*, 2020.
- [2] K. Hashimoto, D. Kim, K. Hauser, and C. K. Liu, “DART: Dynamic Animation and Robotics Toolkit,” *Journal of Open Source Software*, vol. 1, no. 5, p. 65, 2016.
- [3] L. Rossini, E. Mingo Hoffman, S. H. Bang, L. Sentis, and N. G. Tsagarakis, “A Real-Time Approach for Humanoid Robot Walking including Dynamic Obstacles Avoidance,” *IEEE Robotics and Automation Letters*, 2024.